Microsoft Defender for Cloud Apps

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Microsoft Defender for Cloud Apps 6
- What you get . . . 6
- What this delivers . . . 6
- The standard deployment model . . . 7
- Boundary . . . 7
- Effort . . . 7

1. Solution Overview — Microsoft Defender for Cloud Apps 7
- What Microsoft Defender for Cloud Apps is . . . 7
- Why this matters . . . 8
- What this solution covers — the three phases . . . 8
- How it works (the end-to-end flow) . . . 9
- What you can control . . . 9
- Limitations (be honest with buyers) . . . 9
- Connections and authentication used . . . 10
- The standard deployment model . . . 10

2. Requirements Guide — Microsoft Defender for Cloud Apps 10
- 2.1 Defender for Cloud Apps activated . . . 11
- 2.2 Priority SaaS apps and connector credentials . . . 11
- 2.3 Cloud Discovery log source . . . 11
- 2.4 App governance enabled . . . 11
- 2.5 Entra Conditional Access for App Control . . . 11
- 2.6 Pilot user group defined . . . 12
- 2.7 Advanced hunting access (CloudAppEvents) . . . 12
- 2.8 Read-only Graph permissions for the scripts . . . 12
- 2.9 Roles: configurator, facilitator, and read-only attendees . . . 12
- 2.10 Policy posture and enforcement governance . . . 12
- 2.11 PoC evaluation criteria (Phase C) . . . 13
- Checklist . . . 13

Global Requirements — Defender (category-level) 13
- G1. Microsoft 365 / Entra tenant . . . 13
- G2. Defender licensing . . . 14
- G3. Administrative roles (least privilege) . . . 14
- G4. Onboarding reach to endpoints / identity / workloads . . . 14
- G5. Network access to Defender service endpoints . . . 14
- G6. A pilot ring . . . 14

G7. Log Analytics workspace (where required) . . . 14
G8. Response governance . . . 15
3. Implementation Guide — Microsoft Defender for Cloud Apps 15
Naming (set once, reuse verbatim) . . . 15
Phase A — Connect & Govern (onboarding) 16
Step A1 — Assess readiness . . . 16
Step A2 — Connect the priority apps (API connectors) . . . 16
Step A3 — Turn on Cloud Discovery . . . 16
Step A4 — Enable app governance and OAuth app policy (report-only) . . . 17
Step A5 — Create session/access policies via Conditional Access App Control (pilot, report-only) . 17
Step A6 — Add anomaly-detection and activity policies (monitor-first) . . . 17
Step A7 — Validate signal on the pilot . . . 17
Phase B — Enable the team (hands-on labs) 18
Lab 0 — Readiness check . . . 18
Lab 1 — Cloud Discovery & shadow IT . . . 18
Lab 2 — App governance & connected apps . . . 18
Lab 3 — OAuth app risk (approval-gated rehearsal) . . . 19
Lab 4 — Session & access policies, report-only first . . . 19
Lab 5 — Anomaly detection & investigation . . . 19
Lab wrap-up . . . 20
Phase C — Prove value (guided proof-of-concept) 20
Step C1 — Enable an audit-mode policy for the pilot . . . 20
Step C2 — Generate benign test signal . . . 20
Step C3 — Validate and demonstrate the detections . . . 21
Step C4 — Score against your criteria . . . 21
Step C5 — Tear down (clean rollback) . . . 21
Enforce and expand tenant-wide 21
5. Testing Guide — Microsoft Defender for Cloud Apps 22
Part A — Deployment validation (Phase A) 22
A1 — App connectors connected . . . 22
A2 — Cloud Discovery flowing . . . 22
A3 — App governance and OAuth policy active . . . 22
A4 — Session/access policy scoped and report-only . . . 23
A5 — Conditional Access App Control routes sessions . . . 23
A6 — Cloud-app signal streaming . . . 23
A7 — A monitor policy fires (no enforcement) . . . 23

A8 — Enforcement/revoke stays approval-gated . . . 23
Part B — Enablement knowledge-check (Phase B) 23
B-env — Lab environment checks . . . 23
B-know — Per-attendee knowledge-check . . . 24
B-capstone — Live demonstration . . . 24
Part C — Proof-of-concept gates (Phase C) 24
C1 — Audit policy scoped . . . 24
C2 — Benign signal generated . . . 25
C3 — Discovery report populated . . . 25
C4 — Activity policy matched and alert surfaced . . . 25
C5 — Activity-log / hunting visibility . . . 25
C6 — Scorecard produced . . . 25
C7 — Clean teardown . . . 25
6. Troubleshooting Guide — Microsoft Defender for Cloud Apps 26
Cloud apps section missing or blades won't open . . . 26
Connector not connecting . . . 26
No discovery data . . . 26
App governance not inventorying . . . 26
App Control not proxying / session policy has no effect . . . 26
No cloud-app signal . . . 27
`MDCAFULL-OAuthRisk.ps1` lists apps but no permission grants . . . 27
Anomaly-detection alerts absent (Lab 5) . . . 27
PoC policy scope or action wrong . . . 27
PoC signal not generated / discovery report empty . . . 27
PoC policy did not match / no alert surfaced . . . 27
PoC teardown left residue . . . 28
Policy too noisy / false positives . . . 28
Policy enforced or app revoked unexpectedly . . . 28
Script cannot authenticate . . . 28
Portal menu paths don't match . . . 28
7. Best Practices — Microsoft Defender for Cloud Apps 28
Connect & Govern (Phase A) . . . 29
Enable the team (Phase B) . . . 29
Prove value (Phase C) . . . 30
Across all phases . . . 30
Where to go next . . . 30

8. Deliverables Summary — Microsoft Defender for Cloud Apps 31
Documentation 31
Deliverables (PowerShell — Microsoft Graph Security / Entra + KQL) 32
Take-home runbook artifacts (produced during the labs) 35
What the buyer receives 35
Outcome 35
Appendix A — PowerShell Scripts 36

Executive Summary — Microsoft Defender for Cloud Apps

White-label · Defender · Complete lifecycle solution

Most tenants that own Microsoft Defender for Cloud Apps (MDCA) run it as an empty console. The **app connectors** were never added, so the SaaS apps that hold the organization's data send no signal; **Cloud Discovery** was never wired to a log source, so shadow IT is invisible; **app governance** and **OAuth app** policies are off, so risky third-party app consents go unwatched; **session/access policies** via Conditional Access App Control were never created, so there is no real-time control over risky sessions — and even where it is switched on, the admins and SOC analysts have never been trained to drive it and the value was never proven to the people who paid for it. The result is a **cloud access security broker (CASB)** that is licensed but blind, or partly wired and unloved.

This is the **complete Defender for Cloud Apps solution** — one product that takes MDCA the whole way, in three phases:

1. **Connect & Govern** — onboard MDCA end to end: API app connectors, Cloud Discovery, app governance and OAuth app policies, and session/access policies via Conditional Access App Control — piloted first, monitor-first, then expanded.
2. **Enable the team** — five guided, hands-on labs plus a take-home runbook and knowledge-check that take your admins and SOC analysts from "we have it" to "we operate it."
3. **Prove value** — a guided, scoped proof-of-concept that generates **benign** test signal, demonstrates the headline detections, and produces a decision scorecard — cleanly rolled back at the end.

It is written for an administrator with no prior MDCA experience. Every enforcing policy stays in **monitor / report-only** until validated, every disruptive or destructive action (block, revoke OAuth app, suspend session, teardown) is `-WhatIf`**-gated or approval-gated**, and no production access is ever blocked to "test."

What you get

- A plain-language **Solution Overview** of app connectors, Cloud Discovery, app governance, OAuth app policies, and session/access control, and how the whole lifecycle fits together.
- A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (the category-wide ones live in `global-requirements-defender.md`).
- An **Implementation Guide** covering all three phases — connect & govern, the five enablement labs, and the guided PoC — with every object named and reused verbatim.
- Six **deliverables**: read-only PowerShell (Microsoft Graph Security) for readiness, validation, and coverage/OAuth-risk reporting; a read-only OAuth-risk enumerator; a `-WhatIf`-gated PoC helper (enable → benign signal → teardown); and a **KQL advanced-hunting** pack.
- **Testing** (deployment validation + the enablement knowledge-check + the PoC gates) and **Troubleshooting** guides.

What this delivers

A fully connected, team-ready, value-proven Defender for Cloud Apps: connectors streaming from priority SaaS apps, Cloud Discovery surfacing shadow IT, app governance and OAuth policies watching risky consents, session/access policies enforcing real-time control on a pilot scope, a **team that can operate all of it** with a runbook they keep, and a **scorecard** that proves the value — the whole CASB lifecycle in one package.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — connect apps and discovery, scope monitor-first policies to a pilot user group, validate signal and control, enable the team on the same pilot, prove value with a benign PoC, then expand tenant-wide.

Boundary

This solution delivers the **Defender for Cloud Apps pillar itself** — onboarding, enablement, and a proof-of-concept. It is distinct from the unified **Defender XDR** onboarding accelerator (**DEFXDR**) and from the **Cloud-App DLP (Defender for Cloud Apps) — Microsoft Purview** solution that builds the data-loss-prevention program on top of the connected apps — cross-reference those, do not duplicate them. Data-classification, acceptable-use, app-sanctioning, and incident-response determinations remain the customer's compliance, security, and operations functions; this guide provides the technical configuration and the operating skills, not the decisions.

Effort

A competent administrator can complete the pilot-scope connect & govern work and the guided PoC in a focused engagement, run the five enablement labs against the same pilot in a one- to two-day session, then expand tenant-wide on a controlled cadence. Connector rollout scales with the number of SaaS apps to govern. MDCA, app governance, and Conditional Access App Control are license-/preview-gated — **verify current availability in your tenant.**

1. Solution Overview — Microsoft Defender for Cloud Apps

What Microsoft Defender for Cloud Apps is

Microsoft Defender for Cloud Apps (MDCA) is Microsoft's **cloud access security broker (CASB)** and the SaaS-security pillar of Microsoft Defender. It gives four capabilities that together provide **cloud app security**:

- **App connectors (API-based)** — deep, out-of-band integrations with major SaaS apps (Microsoft 365, and third-party apps such as Salesforce, ServiceNow, Google Workspace, AWS, Box) that pull activity logs, files, accounts, and settings so MDCA can detect threats and apply file/governance policies.
- **Cloud Discovery** — analysis of traffic logs (from Defender for Endpoint's built-in feed, a log collector, a secure web gateway, or an uploaded snapshot log) to surface **shadow IT**: which cloud apps are in use, by whom, and at what risk.

- **App governance and OAuth app policies** — visibility and policy over the **third-party OAuth apps** users consent to in the tenant, catching over-privileged or malicious app grants.
- **Session and access policies (via Conditional Access App Control)** — real-time, in-line control of user sessions (block download, restrict copy/paste, require read-only) proxied through MDCA and triggered by Entra Conditional Access.

On top of these, MDCA runs **anomaly-detection** policies (impossible travel, mass download, unusual admin activity, ransomware-like activity) that surface alerts into Defender XDR **Incidents & alerts** and the `CloudAppEvents` advanced-hunting table. Most administration is in the unified Defender portal at `security.microsoft.com` under **Cloud apps**; Conditional Access App Control is wired in Entra.

Why this matters

Organizational data has moved into SaaS, but the security controls often have not. Without **connectors**, MDCA cannot see what happens inside the apps that hold the data. Without **Cloud Discovery**, shadow IT stays invisible until it causes an incident. Without **app governance / OAuth policies**, a single malicious or over-permissioned OAuth grant can exfiltrate mail and files with no password ever stolen. Without **session/access policies**, there is no way to allow a risky sign-in but constrain what it can do. And even a fully wired MDCA delivers nothing if the team cannot drive it or if the value was never proven to the budget owner. This solution turns the empty (or half-configured) CASB into a working, team-operated, value-proven one — validated on a pilot scope before it can affect production access.

What this solution covers — the three phases

This is a **complete lifecycle** product. It delivers the whole arc in one package:

Phase A — Connect & Govern (onboarding)

Onboard MDCA end to end: add **app connectors** for the priority SaaS apps, turn on **Cloud Discovery**, enable **app governance** and an **OAuth app policy** (report-only), and create **session/access policies** via **Conditional Access App Control** — every enforcing policy starts monitor / report-only, scoped to a **pilot user group**, and is validated before it enforces or expands.

Phase B — Enable the team (hands-on labs)

Upskill your admins and SOC analysts with **five guided labs**, a take-home runbook, and a knowledge-check — all run against the same pilot scope, **read-only or report-only**:

- **Lab 1 — Cloud Discovery & shadow IT.** Run/read a discovery report, rank apps by risk, tag Sanctioned/Unsanctioned.
- **Lab 2 — App governance & connected apps.** Explore a connected app's activity; use app governance to review app behavior and API usage.
- **Lab 3 — OAuth app risk.** Inventory OAuth apps, read permissions and risk, rehearse the **approval-gated** disable/revoke.
- **Lab 4 — Session & access policies.** Author a Conditional Access App Control session policy in **monitor/report-only**; understand the enforce path.
- **Lab 5 — Anomaly detection & investigation.** Read anomaly-detection alerts, triage with the entity/activity log and advanced hunting, rehearse response.

Phase C — Prove value (guided proof-of-concept)

Run a **scoped, safe** PoC on the pilot users: enable an **audit-mode** activity policy, generate **benign** test signal (Microsoft's own **sample Cloud Discovery log** plus benign, policy-matching activity by a test account — never real exfiltration or attacks), demonstrate the detections, **score** the result against your evaluation criteria, and **cleanly tear the PoC back down**.

How it works (the end-to-end flow)

1. **App connectors** are added under **Cloud apps** → **Connected apps** → **App connectors** (Microsoft 365 first, then priority third-party apps); each pulls activity, files, and accounts via the app's API (out of band — no user impact).

2. **Cloud Discovery** is enabled — the simplest path is the built-in **Defender for Endpoint integration** so onboarded devices feed discovery automatically; a **log collector**, SWG feed, or uploaded snapshot log is used where MDE is not the source.

3. **App governance** is turned on and an **OAuth app policy** is created to flag/limit risky third-party app consents (over-privileged scopes, unverified publishers, unusual consent patterns).

4. **Policies** (activity, file, anomaly-detection, session, access) start **monitor / report-only**, scoped to a **pilot user group**, so behavior is observed before anything enforces.

5. **Session and access policies** are wired through **Conditional Access App Control** in Entra — a CA policy routes the pilot users' sessions to MDCA, which then applies the session controls.

6. **The team is enabled** on that same pilot with the five labs and a knowledge-check.

7. **Value is proven** with a benign, audit-mode PoC and a scorecard.

8. **Validation** confirms `CloudAppEvents` and discovery data stream and that scoped monitor policies generate the expected alerts — then enforcing controls are broadened from the pilot only after they are validated.

What you can control

• **Which apps are connected** and the order they onboard (Microsoft 365 first, then priority third-party apps).

• **Cloud Discovery source** — MDE integration, log collector, SWG feed, or an uploaded snapshot log.

• **OAuth app governance** — which app consents are flagged, limited, or revoked (revocation approval-gated).

• **Policy posture** — monitor/report-only first, scoped to a pilot user group, before enforcing.

• **Session/access controls** — what a proxied session may do (download, copy, print, read-only) and for whom.

• **PoC scope, benign signal, and scorecard criteria** — exactly which pilot users, which capabilities to prove, and the pass/fail bar.

Limitations (be honest with buyers)

• MDCA and app governance are **license-gated** (Defender for Cloud Apps; app governance may require a specific plan). Onboarding without the plan fails or limits features — *verify current availability in your tenant.*

• **Session/access policies require Conditional Access App Control**, which depends on Entra ID P1/P2 Conditional Access and reverse-proxy session routing — *verify current licensing and supported apps.*

• **Cloud Discovery** depends on a log source; without the MDE feed, a log collector, or an uploaded log there is no shadow-IT visibility.

• Discovery, connectors, and hunting tables **populate over time** — allow warm-up after onboarding and after generating signal.

• All PoC test signal is **benign and simulated** (sample log + benign policy-matching activity) — it proves the visibility/policy pipeline, not resilience to a real exfiltration.

• MDCA controls cloud-app access and data movement; it does not replace endpoint EDR, email protection, identity hardening, or the customer's DLP-classification and acceptable-use decisions.

Connections and authentication used

• The **unified Defender portal** (`security.microsoft.com` → **Cloud apps**) for connectors, discovery, app governance, OAuth apps, policies, alerts, and advanced hunting. *Some MDCA settings are still consolidating from the classic portal — verify current navigation in your tenant.*

• **Entra** (`entra.microsoft.com`) for **Conditional Access App Control** that routes sessions to MDCA.

• **Microsoft Graph Security** and the **Microsoft 365 Defender** APIs (via the `Microsoft.Graph` PowerShell modules) for read-only discovery, validation, reporting, and advanced-hunting (`CloudAppEvents`, `OAuthAppInfo`, `AlertInfo`) with delegated **read** scopes.

• Least-privilege roles: **Security Administrator** for configuration and the PoC enable/teardown, **Security Reader** for read-only work and lab attendees, and **Global Administrator** only for one-time first-service activation and initial OAuth consent.

The standard deployment model

> ***Assess readiness (license, roles, apps, discovery source) → connect apps and enable discovery/app-governance → scope monitor-first policies to a pilot user group and wire Conditional Access App Control → enable the team on the pilot → prove value with a benign, audit-mode PoC and a scorecard → validate signal and control → expand tenant-wide.***
> *Repeatable pattern: **Assess → Configure → Pilot → Validate → Expand**, with enforcing/revoke/teardown actions kept `-WhatIf`- or approval-gated until trusted.*

2. Requirements Guide — Microsoft Defender for Cloud Apps

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant, licensing, admin roles, onboarding reach, network access, pilot ring, Log Analytics workspace, response governance) are in

`global-requirements-defender.md` — do not repeat them; the items below are **specific to this solution**. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

2.1 Defender for Cloud Apps activated

Description: MDCA initialized for the tenant under `security.microsoft.com` → **Cloud apps**. **Why:** Connectors, discovery, app governance, and policies only appear after first-time activation. **Verify:** Sign in as a Security Administrator and confirm **Cloud apps** → **Cloud discovery / Connected apps / Policies** load. Run `MDCAFULL-Discovery.ps1`. **Implement-if-missing:** Complete the one-time MDCA activation (Global Administrator required for first-service activation only). *Confirm the exact plan — verify current availability (references `global-requirements-defender.md` G2).*

2.2 Priority SaaS apps and connector credentials

Description: The list of SaaS apps to connect first (Microsoft 365, then priority third-party apps) and the admin credentials/consent each connector needs. **Why:** App connectors are API integrations; each requires an admin consent or API credential for that app. **Verify:** A prioritized app list exists; `MDCAFULL-Discovery.ps1` reports which connectors are already added. **Implement-if-missing:** Add the connector for the top app(s) under **Cloud apps** → **Connected apps** → **App connectors** and complete each app's consent flow. Name the effort `MDCA App Connectors – Priority SaaS – Prod`.

2.3 Cloud Discovery log source

Description: A traffic-log source for Cloud Discovery — the built-in **Defender for Endpoint integration**, a **log collector**, a secure web gateway feed, or (for the PoC) an uploaded **sample log**. **Why:** Without a log source there is no shadow-IT discovery; the MDE feed is the lowest-friction option if endpoints are onboarded to MDE. **Verify:** `security.microsoft.com` → **Cloud apps** → **Cloud discovery** → **Settings**; confirm a source is configured. `MDCAFULL-Discovery.ps1` reports the discovery source. **Implement-if-missing:** Turn on the **Defender for Endpoint** → **Cloud Apps** integration (if devices are MDE-onboarded — see the adjacent MDE onboarding accelerator), deploy a log collector for a SWG, or upload a snapshot log for the PoC/labs.

2.4 App governance enabled

Description: App governance (OAuth app monitoring and policy) turned on for the tenant. **Why:** Risky third-party OAuth app consents are a major exfiltration path; app governance is what watches them, and Labs 2/3 use it. **Verify:** `security.microsoft.com` → **Cloud apps** → **App governance**; confirm it is enabled. `MDCAFULL-Discovery.ps1` reports app-governance status. *App governance may require a specific plan — verify current availability.* **Implement-if-missing:** Enable app governance and allow it to inventory the tenant's OAuth apps before creating policies. If it cannot be enabled, Lab 3 can still use the **OAuth apps** inventory read-only.

2.5 Entra Conditional Access for App Control

Description: Entra Conditional Access (P1/P2) available to route pilot users' sessions to MDCA for **Conditional Access App Control**. **Why:** Session and access policies are enforced by proxying sessions,

triggered by a Conditional Access policy (Phase A Step 5 and Lab 4). **Verify:** `entra.microsoft.com` → **Protection** → **Conditional Access**; confirm CA is available and a policy can target the pilot users. *Verify current App Control licensing and supported apps.* **Implement-if-missing:** Confirm Entra ID P1/P2, then plan the CA policy `MDCA Session Control – Pilot Users – Report`.

2.6 Pilot user group defined

Description: A named pilot user group `MDCA Pilot Users – Pilot Ring – Pilot` to scope monitor-first policies, session control, the enablement labs, and the PoC. **Why:** Policies (especially session/access) are validated on a pilot group before tenant-wide enforcement (per `global-requirements-defender.md` G6); the labs and PoC also run against this scope. **Verify:** The group exists in Entra; `MDCAFULL-Validate.ps1 -Check Policy` checks the scope. **Implement-if-missing:** Create the group `MDCA Pilot Users – Pilot Ring – Pilot` (include the test account used to generate benign PoC activity) before scoping any policy.

2.7 Advanced hunting access (CloudAppEvents)

Description: The team can open **Advanced hunting** and query the `CloudAppEvents` table. **Why:** Validation, several labs, and the PoC run the KQL pack (`MDCAFULL-LabQueries.kql`) against `CloudAppEvents` and related tables. **Verify:** Open **Hunting** → **Advanced hunting**; run `CloudAppEvents | take 5`; confirm rows return. **Implement-if-missing:** Grant the team a role including advanced-hunting read (e.g. **Security Reader** plus hunting). *Confirm role mapping in your tenant.*

2.8 Read-only Graph permissions for the scripts

Description: The PowerShell deliverables authenticate to Microsoft Graph with **read-only** scopes (e.g. `Organization.Read.All`, `Directory.Read.All`, `Application.Read.All`, `SecurityEvents.Read.All`, `Policy.Read.All`). **Why:** `MDCAFULL-Discovery.ps1`, `MDCAFULL-Validate.ps1`, `MDCAFULL-Report.ps1`, and `MDCAFULL-OAuthRisk.ps1` read licensing, roles, connectors, alerts, service principals, and OAuth grants — they change nothing. **Verify:** `Connect-MgGraph -Scopes "SecurityEvents.Read.All","Application.Read.All","Directory.Read.All"` consents successfully. **Implement-if-missing:** Have a Global Administrator consent the delegated read scopes once, or register a read-only app. Never grant write/consent-management scopes for the read-only work.

2.9 Roles: configurator, facilitator, and read-only attendees

Description: A **Security Administrator** for Phase A configuration and the PoC enable/teardown; a **facilitator** with an MDCA admin role for the single report-only lab policy; **attendees** with **Security Reader** (plus hunting) to follow along read-only. **Why:** Least privilege — only the admin/facilitator authors policies; everyone else learns and validates read-only and rehearses enforce/revoke in **simulate/approval-gated** mode. **Verify:** Confirm the roles are held by the right people before starting. **Implement-if-missing:** Assign roles before the engagement; reserve enforce/revoke/teardown for the approval-gated / `-WhatIf`-gated steps.

2.10 Policy posture and enforcement governance

Description: Agreement to start every policy in **monitor / report-only**, keep the PoC in **audit mode**, and keep enforcing/destructive actions (block, revoke OAuth app, suspend session, PoC teardown) approval-gated or `-WhatIf`-gated. **Why:** Enforcing policies and destructive actions can disrupt legitimate work; the organization decides the guardrails (per `global-requirements-defender.md` G8). **Verify:** A documented policy posture exists; policies are report-only first. `MDCAFULL-Validate.ps1 -Check Policy` reports the posture. **Implement-if-missing:** Create policies in report-only mode; *enforcement, OAuth-revocation, and teardown decisions are the security team's — this guide covers the technical configuration.*

2.11 PoC evaluation criteria (Phase C)

Description: A written set of pass/fail evaluation criteria for the proof-of-concept. **Why:** The PoC scorecard (`MDCAFULL-PoC.ps1 -Action Score`) maps each observation to your criteria — agreeing them first keeps the verdict objective. **Verify:** A `criteria.json` (or equivalent) exists listing the capabilities to prove (discovery populated, policy matched, alert surfaced, activity visible). **Implement-if-missing:** Draft the criteria with stakeholders before running the PoC.

Checklist

- MDCA activated — `MDCAFULL-Discovery.ps1`
- Priority SaaS apps + connector credentials (`MDCA App Connectors – Priority SaaS – Prod`)
- Cloud Discovery log source (MDE integration / log collector / SWG / sample log)
- App governance enabled
- Entra Conditional Access available for App Control
- Pilot user group `MDCA Pilot Users – Pilot Ring – Pilot` defined (with a test account)
- Advanced hunting access to `CloudAppEvents`
- Read-only Graph scopes consented
- Roles: Security Administrator / facilitator / read-only attendees
- Policy posture (report-only first, enforce/revoke/teardown gated) documented
- PoC evaluation criteria written

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the

SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Microsoft Defender for Cloud Apps

Follow these steps **in order**. Most configuration is in the **unified Defender portal** (`https://security.microsoft.com` → **Cloud apps**); Conditional Access App Control is wired in the **Entra admin center** (`https://entra.microsoft.com`). Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test. *Some MDCA settings are still consolidating from the classic portal — verify current navigation in your tenant; the Troubleshooting Guide lists fallbacks.*

> ***Model:*** *Connect the apps and discovery, scope every policy* ***monitor / report-only*** *to a* ***pilot user group*** *first,* ***enable the team*** *on that same pilot,* ***prove value*** *with a benign audit-mode PoC, validate, and only then enforce and expand. Keep enforcing actions (block, revoke OAuth app, suspend session) and the PoC teardown* `-WhatIf-` *or approval-gated throughout. Never block production access to "test."*

Naming (set once, reuse verbatim)

Phase A — Connect & Govern

- Pilot user group (Entra): `MDCA Pilot Users – Pilot Ring – Pilot`
- App connectors effort: `MDCA App Connectors – Priority SaaS – Prod`
- OAuth app policy: `MDCA OAuth App Risk – Tenant – Report`
- Session policy (via CA App Control): `MDCA Session Control – Pilot Users – Report`
- Access policy: `MDCA Access Control – Pilot Users – Report`
- Conditional Access policy (Entra): `MDCA Session Control – Pilot Users – Report`
- Advanced-hunting saved query (signal): `MDCA Signal Validation – Tenant – Validate`

Phase B — Enablement labs (take-home artifacts)

- Saved hunting queries: `Discovery-ShadowIT – CloudApps – Pilot`, `OAuthApps – CloudApps – Pilot`, `Anomaly-Investigation – CloudApps – Pilot`

• Shortlists: `ShadowIT-Shortlist – CloudApps – Pilot`, `OAuthRisk-Shortlist – CloudApps – Pilot`

• Investigation doc: `Investigation – CloudApps – Pilot`

Phase C — Proof-of-concept

• Audit-mode activity policy: `MDCA PoC Activity Policy – Pilot Users – Audit`

• Discovery snapshot report: `MDCA PoC Discovery – Pilot – Report`

• Scorecard output file: `MDCA PoC Scorecard – Pilot Users – Validate`

• Test-signal tag (used in notes/hunting): `MDCA-PoC-TESTSIGNAL`

Phase A — Connect & Govern (onboarding)

Step A1 — Assess readiness

1. Sign in to `security.microsoft.com` as a **Security Administrator**.

2. Run `MDCAFULL-Discovery.ps1` to capture the current state: MDCA activation, plan hints, admin roles, existing app connectors, Cloud Discovery source, app-governance status, and Conditional Access availability.

3. In **Entra** → **Groups**, create the pilot user group `MDCA Pilot Users – Pilot Ring – Pilot` and add the pilot/test users (including the account used to generate benign PoC activity in Phase C).

Step A2 — Connect the priority apps (API connectors)

1. `security.microsoft.com` → **Cloud apps** → **Connected apps** → **App connectors** → **Connect an app**.

2. Connect **Microsoft 365** first, then the priority third-party apps as `MDCA App Connectors – Priority SaaS – Prod`; complete each app's admin consent / API-credential flow.

3. Confirm each connector shows **Connected** and begins pulling activity (connectors work out of band — no user impact).

4. Run `MDCAFULL-Validate.ps1 -Check Connectors` to confirm the connectors are connected and streaming.

Step A3 — Turn on Cloud Discovery

1. `security.microsoft.com` → **Cloud apps** → **Cloud discovery** → **Settings**.

2. Enable the **Microsoft Defender for Endpoint integration** so MDE-onboarded devices feed discovery automatically (preferred). Where MDE is not the source, configure a **log collector** for the secure web gateway.

3. Confirm discovery data appears under **Cloud apps** → **Cloud discovery** → **Dashboard**.

4. Run `MDCAFULL-Validate.ps1 -Check Discovery` to confirm discovery data is flowing.

Step A4 — Enable app governance and OAuth app policy (report-only)

1. `security.microsoft.com` → **Cloud apps** → **App governance** — enable it and let it inventory the tenant's OAuth apps. *App governance may require a specific plan — verify current availability.*

2. Create the OAuth app policy `MDCA OAuth App Risk – Tenant – Report` in **report-only** to flag over-privileged scopes, unverified publishers, and unusual consent patterns — **no auto-revoke** yet.

3. Run `MDCAFULL-OAuthRisk.ps1` to list the current risky OAuth apps (read-only) and `MDCAFULL-Report.ps1 -Section OAuth` for the shortlist template.

4. *Revoking an app consent is disruptive and stays approval-gated — the decision is the security team's.*

Step A5 — Create session/access policies via Conditional Access App Control (pilot, report-only)

1. In **Entra** → **Protection** → **Conditional Access** → **New policy**, create `MDCA Session Control – Pilot Users – Report`: assign **Users =** `MDCA Pilot Users – Pilot Ring – Pilot`, target cloud apps, and under **Session** choose **Use Conditional Access App Control** → **Use custom policy** (routes sessions to MDCA). *Verify current App Control licensing and supported apps.*

2. In `security.microsoft.com` → **Cloud apps** → **Policies** → **Policy management**, create the session policy `MDCA Session Control – Pilot Users – Report` (e.g. monitor downloads on unmanaged devices) in **monitor** mode, and the access policy `MDCA Access Control – Pilot Users – Report` in **monitor** mode — both scoped to the pilot users.

3. Run `MDCAFULL-Validate.ps1 -Check Policy` to confirm the policies exist, are report-only, and are scoped to the pilot group.

Step A6 — Add anomaly-detection and activity policies (monitor-first)

1. `security.microsoft.com` → **Cloud apps** → **Policies** — confirm the built-in **anomaly detection** policies are on (impossible travel, mass download, risky OAuth), tuned to the pilot scope where possible.

2. Add any needed **activity** or **file** policies in **report-only** first.

3. Run `MDCAFULL-Report.ps1 -Section Policies` to capture the current policy inventory and posture.

Step A7 — Validate signal on the pilot

1. In **Hunting** → **Advanced hunting**, paste the **signal-validation** section of `MDCAFULL-LabQueries.kql`, run it, and **Save as** `MDCA Signal Validation – Tenant – Validate`. It confirms `CloudAppEvents` (and OAuth app data) stream from the connectors and discovery. `MDCAFULL-Validate.ps1 -Check Signal` cross-checks read-only.

2. As a pilot user, perform a benign action a monitor policy watches (e.g. a download on an unmanaged device) and confirm the expected **alert** appears in **Incidents & alerts** — without enforcement blocking anything.

Phase B — Enable the team (hands-on labs)

Run the five labs **in order** against the same `MDCA Pilot Users – Pilot Ring – Pilot` scope. Every step is **read-only or report-only**; the one configuration action (the report-only session policy, already created in Step A5) and any disable/enforce are **facilitator-driven and approval-gated**. The take-home output is the saved queries, shortlists, and investigation doc.

Lab 0 — Readiness check

1. Sign in as the **facilitator** and open the **Cloud apps** section.
2. Confirm a **Cloud Discovery** dashboard with data, at least one **Connected app**, and **App governance** enabled (from Phase A).
3. Run `MDCAFULL-Discovery.ps1` and `MDCAFULL-Validate.ps1` — confirm green. **Pass gate:** discovery + validate green; Cloud Discovery, a connector, and app governance all visible.

Lab 1 — Cloud Discovery & shadow IT

Goal: run/read a discovery report, rank apps by risk, tag Sanctioned/Unsanctioned.

1. Open **Cloud apps** → **Cloud Discovery** → **Dashboard**; review total apps, traffic, and top apps by users/traffic.
2. Open **Discovered apps**; sort by **Risk score**; discuss what drives the score (compliance, security, legal factors).
3. Pick 3–5 high-risk, high-usage apps. For a clearly unsanctioned one, use the **tag** action to mark **Unsanctioned** (facilitator; note that tagging alone is informational unless a block is wired via a proxy/appliance). Mark an approved one **Sanctioned**.
4. In **Advanced hunting**, paste the **shadow-IT / discovery** section of `MDCAFULL-LabQueries.kql`; run it; **Save as** `Discovery-ShadowIT – CloudApps – Pilot`.
5. Build the `ShadowIT-Shortlist – CloudApps – Pilot` — top risky apps and a recommended disposition (sanction / restrict / block-later). **App-sanctioning decisions are the customer's** — the shortlist is a recommendation.
6. **Pass gate:** each attendee can explain the risk score and name the top shadow-IT app and its proposed disposition.

Lab 2 — App governance & connected apps

Goal: examine a connected app's activity and use app governance to read app behavior.

1. Open **Cloud apps** → **Connected apps**; open the **Microsoft 365** connector; review **Activity log**, **Files**, and **Accounts** tabs.
2. Open **Cloud apps** → **App governance**; review the app inventory: permissions, data access, usage, and any **risk factors / alerts**.
3. Pick one app with notable permissions/usage; walk its behavior (API calls, data accessed, users). Discuss "normal" vs "suspicious."

4. In **Advanced hunting**, run the **connected-app activity** section of `MDCAFULL-LabQueries.kql` against `CloudAppEvents`; review results.

5. **Pass gate:** each attendee can navigate a connected app's activity log and state one app-governance risk indicator and what it means.

Lab 3 — OAuth app risk (approval-gated rehearsal)

Goal: inventory OAuth apps, read consent/permission risk, and rehearse disable/revoke.

1. Open the **OAuth apps** inventory (**Cloud apps** → **OAuth apps**, and/or **App governance**). Review third-party apps granted access, their **permission level**, **community use**, and **publisher verification**.

2. Run `MDCAFULL-OAuthRisk.ps1` — a **read-only** enumeration of service principals and OAuth grants with risk hints (high-privilege scopes, unverified publisher, broad consent). It changes nothing.

3. In **Advanced hunting**, run the **OAuth apps** section of `MDCAFULL-LabQueries.kql`; **Save as** `OAuthApps – CloudApps – Pilot`.

4. Build the `OAuthRisk-Shortlist – CloudApps – Pilot` — the riskiest apps by consent scope and verification status.

5. **Response rehearsal (approval-gated):** discuss how you would **disable / revoke consent** for a risky app (from the OAuth apps inventory or Entra Enterprise applications). **Describe/queue for approval; do not revoke a production app** — revoking consent can break a legitimate integration. Revocation is a change the customer's app owners/security approve.

6. **Pass gate:** each attendee names two OAuth risk indicators and states who must approve a consent revocation.

Lab 4 — Session & access policies, report-only first

Goal: understand the Conditional Access App Control session policy in **monitor/report-only** mode and the enforce path.

1. Confirm the app is routed through **Conditional Access App Control** (the CA policy `MDCA Session Control – Pilot Users – Report` from Step A5). *Verify current setup in your tenant.*

2. Open the session policy `MDCA Session Control – Pilot Users – Report` in **Cloud apps** → **Policies** → **Policy management**; confirm it is **Monitor only** (not Block), scoped to the pilot group.

3. Generate benign activity (a download in the pilot session) and observe it **logged, not blocked**.

4. Discuss the enforce path: how you would change **Monitor only** → **Block download** after validation, staged to a ring, with a rollback. **Enforcement is staged and change-approved — not done during training.**

5. **Pass gate:** each attendee can describe the monitor→enforce progression and the rollback.

Lab 5 — Anomaly detection & investigation

Goal: read anomaly-detection alerts, triage, and rehearse response.

1. Open **Incidents & alerts** → **Alerts**; filter **Service source = Defender for Cloud Apps**. Review anomaly-detection alert types (impossible travel, mass download, unusual admin activity,

ransomware-like activity).

2. Take one alert (or a `CloudAppEvents` hit) through triage: which user/app, what activity, is it expected, what would escalate it.

3. In **Advanced hunting**, run the **anomaly investigation** section of `MDCAFULL-LabQueries.kql`; **Save as** `Anomaly-Investigation – CloudApps – Pilot`. Cross-check the user's activity across apps.

4. Capture findings in `Investigation – CloudApps – Pilot`: what happened, which user/app, confidence, recommended action.

5. **Response rehearsal (approval-gated):** review response options (suspend user, require sign-in, mark app unsanctioned/block via proxy). **Rehearse and queue for approval; do not action a production user. Incident-response decisions are the SOC/security team's.**

6. **Pass gate:** a completed investigation doc plus each attendee stating the fitting response and who approves it.

Lab wrap-up

1. Confirm the take-home artifacts: the three saved hunting queries, the report-only session policy, the two shortlists, and the investigation doc.

2. Run the **knowledge-check** in `05-testing.md` (Part B).

Phase C — Prove value (guided proof-of-concept)

Run the PoC on the same pilot users, **benign and audit-mode only**. All test signal is Microsoft's sample discovery log and benign policy-matching activity — **never real exfiltration or attacks**. Enforcement stays **off**; every change is previewed with `-WhatIf` first.

Step C1 — Enable an audit-mode policy for the pilot

1. Run `MDCAFULL-PoC.ps1 -Action Enable -WhatIf` to preview (changes nothing): plan, roles, discovery source, connectors, and whether the pilot group exists.

2. `security.microsoft.com` → **Cloud apps** → **Policies** → **Policy management** → **Create policy** → **Activity policy**. Name it `MDCA PoC Activity Policy – Pilot Users – Audit`. Scope the filter to `MDCA Pilot Users – Pilot Ring – Pilot`, choose a benign, repeatable trigger (e.g. a specific administrative activity or a download threshold), and set **Governance actions = none** (audit only — no block, no session control).

3. Re-run `MDCAFULL-PoC.ps1 -Action Enable` (without `-WhatIf`) to guide/mark the audit-policy creation scoped to the pilot group.

Step C2 — Generate benign test signal

1. Run `MDCAFULL-PoC.ps1 -Action Signal` (or follow its printed steps). It performs only **benign, documented** actions tagged `MDCA-PoC-TESTSIGNAL`:

• **Cloud Discovery:** `Cloud apps → Cloud discovery → Actions → Create Cloud Discovery snapshot report` → upload Microsoft's **sample Cloud Discovery log**. This produces the Shadow IT report `MDCA PoC Discovery – Pilot – Report` from safe, sample data.

• **Policy-matching activity:** as the test account, perform the **benign** action the audit policy watches for (e.g. download a few non-sensitive files from a monitored app). Nothing is exfiltrated.

1. **Do not** exfiltrate real data, connect a malicious app, or grant a risky OAuth consent. The sample log and benign activity are sufficient.

2. Note the time of each signal for the scorecard latency comparison.

Step C3 — Validate and demonstrate the detections

1. Run `MDCAFULL-Validate.ps1 -Check PoC` — it confirms (read-only): the discovery report populated, the audit policy `MDCA PoC Activity Policy – Pilot Users – Audit` matched, an alert surfaced (`AlertInfo`), and activity appears in `CloudAppEvents`.

2. Walk the demo in the portal: `Cloud apps → Cloud discovery → Dashboard` (the Shadow IT report), `Cloud apps → Activity log` (the benign activity), `Cloud apps → Alerts` / **Incidents & alerts** (the policy match).

3. Confirm **no governance action** was taken (audit only) and no production user affected.

Step C4 — Score against your criteria

1. Run `MDCAFULL-PoC.ps1 -Action Score -CriteriaPath .\criteria.json -OutFile "MDCA PoC Scorecard – Pilot Users – Validate.json"`. It maps each observation (discovery populated, policy matched, alert surfaced, activity visible) to your pass/fail criteria and writes the scorecard.

2. Review the scorecard with stakeholders — the decision artefact.

Step C5 — Tear down (clean rollback)

1. Run `MDCAFULL-PoC.ps1 -Action Teardown -WhatIf` first to preview, then for real.

2. Teardown: **delete** the policy `MDCA PoC Activity Policy – Pilot Users – Audit`, **delete** the discovery report `MDCA PoC Discovery – Pilot – Report`, **remove** any optional PoC connector, and **resolve** the `MDCA-PoC-TESTSIGNAL` alerts. Keep the scorecard as your record. *The teardown is a destructive action — it stays `-WhatIf`-gated by default.*

Enforce and expand tenant-wide

1. **Policies:** after validation, move the pilot session/access policies from **monitor** to **enforce** for the pilot group, confirm no false positives, then broaden the CA policy `MDCA Session Control – Pilot Users – Report` scope in waves toward the full tenant.

2. **Connectors:** add connectors for the remaining SaaS apps.

3. **OAuth governance:** review flagged apps with the security team; only then consider approval-gated revocation for the riskiest.

4. **Discovery:** expand the discovery source coverage (more MDE devices / log-collector sources).

5. **Team:** use the labs and runbook to onboard new analysts.

6. Re-baseline with `MDCAFULL-Report.ps1 -SaveBaseline` and track coverage/OAuth-risk drift each cycle.

Order of operations

`Assess → connect apps (M365 first) → turn on Cloud Discovery → enable app governance + OAuth report policy → create session/access policies via CA App Control (pilot, report-only) → add anomaly/activity policies → validate signal → run the five enablement labs → run the benign audit-mode PoC (enable → signal → validate → score → teardown) → enforce + expand`

5. Testing Guide — Microsoft Defender for Cloud Apps

Testing spans all three phases: **Part A** validates the deployment (Phase A), **Part B** is the enablement knowledge-check (Phase B), and **Part C** is the proof-of-concept gate (Phase C). Each test has a **pass/fail gate**; do not enforce or expand until the pilot passes. Objects are referenced by the exact names from the Implementation Guide. Run the scripts by name to validate the environment.

Part A — Deployment validation (Phase A)

A1 — App connectors connected

Procedure: Run `MDCAFULL-Validate.ps1 -Check Connectors`. In `security.microsoft.com` → **Cloud apps** → **Connected apps** → **App connectors**, confirm each connector in `MDCA App Connectors – Priority SaaS – Prod` shows **Connected**. **Pass:** Microsoft 365 (and each priority app) is connected and pulling activity. **Fail:** A connector shows an error/disconnected — see Troubleshooting *Connector not connecting*.

A2 — Cloud Discovery flowing

Procedure: Run `MDCAFULL-Validate.ps1 -Check Discovery`. In **Cloud apps** → **Cloud discovery** → **Dashboard**, confirm discovered apps and traffic appear. **Pass:** Discovery data (apps, users, traffic) is visible from the configured source. **Fail:** Dashboard empty — see Troubleshooting *No discovery data*.

A3 — App governance and OAuth policy active

Procedure: Run `MDCAFULL-OAuthRisk.ps1` and `MDCAFULL-Report.ps1 -Section OAuth`. In **Cloud apps** → **App governance**, confirm OAuth apps are inventoried and `MDCA OAuth App Risk – Tenant – Report` is present and report-only. **Pass:** App governance is on, OAuth apps are inventoried, and the report policy flags risky apps without revoking. **Fail:** App governance off or no inventory — see

Troubleshooting *App governance not inventorying*.

A4 — Session/access policy scoped and report-only

Procedure: Run `MDCAFULL-Validate.ps1 -Check Policy`. In **Cloud apps** → **Policies**, confirm `MDCA Session Control – Pilot Users – Report` and `MDCA Access Control – Pilot Users – Report` exist, are **monitor/report-only**, and are scoped to `MDCA Pilot Users – Pilot Ring – Pilot`. **Pass:** Both policies exist, are report-only, and target only the pilot users. **Fail:** Policy missing, enforcing, or wrongly scoped — re-create per Implementation Step A5.

A5 — Conditional Access App Control routes sessions

Procedure: As a pilot user, sign in to a targeted app; confirm the session is proxied (the MDCA session banner/URL suffix appears) per the CA policy `MDCA Session Control – Pilot Users – Report`. **Pass:** The pilot user's session is routed through MDCA (App Control active) without blocking access. **Fail:** Session not proxied — see Troubleshooting *App Control not proxying*.

A6 — Cloud-app signal streaming

Procedure: Run `MDCAFULL-Validate.ps1 -Check Signal` and the saved query `MDCA Signal Validation – Tenant – Validate`. Confirm `CloudAppEvents` returns recent rows from the connectors/discovery. **Pass:** `CloudAppEvents` streams recent activity. **Fail:** Empty — see Troubleshooting *No cloud-app signal*.

A7 — A monitor policy fires (no enforcement)

Procedure: As a pilot user, perform a benign action a monitor policy watches (e.g. a download on an unmanaged device). In **Incidents & alerts**, confirm the expected alert appears — and access is **not** blocked. **Pass:** The monitor policy generates the alert; the pilot user is not disrupted. **Fail:** No alert or access was blocked — re-check the policy mode/scope (Implementation Step A5).

A8 — Enforcement/revoke stays approval-gated

Procedure: Run `MDCAFULL-Report.ps1 -Section Policies` and confirm no policy auto-blocks or auto-revokes during the pilot. **Pass:** All enforcing/revoke actions require approval; nothing auto-executes on the pilot. **Fail:** An enforcing action is live before agreed — reset to report-only and confirm with the security team.

Part B — Enablement knowledge-check (Phase B)

B-env — Lab environment checks

- **After Lab 3:** run `MDCAFULL-OAuthRisk.ps1`; confirm it lists OAuth apps with risk hints and the `OAuthRisk-Shortlist – CloudApps – Pilot` was captured.

• **After Lab 4:** in **Cloud apps** → **Policies**, confirm `MDCA Session Control – Pilot Users – Report` is **Monitor only** (not Block).

• **After Labs 1/3/5:** in **Advanced hunting** → **Queries**, confirm `Discovery-ShadowIT – CloudApps – Pilot`, `OAuthApps – CloudApps – Pilot`, and `Anomaly-Investigation – CloudApps – Pilot` exist and run without error.

B-know — Per-attendee knowledge-check

Score each attendee; passing = correct on at least 8 of 10.

1. **CASB basics:** What four questions does a CASB answer, and which MDCA feature answers each?
2. **Cloud Discovery:** What feeds Cloud Discovery, and what drives an app's risk score?
3. **Shadow IT:** What does tagging an app **Unsanctioned** do, and what else is needed to actually block it?
4. **Connectors vs discovery:** How is an **app connector** different from **Cloud Discovery**?
5. **App governance:** Name two risk indicators app governance surfaces on an app.
6. **OAuth risk:** Give two OAuth app risk factors and why each is dangerous *(e.g. broad consent scopes, unverified publisher)*.
7. **Session policy:** Why author a session policy **report-only / monitor** first, and what is the enforce path?
8. **Anomaly detection:** Name three anomaly-detection alert types and one triage question for each.
9. **Hunting:** Which advanced-hunting table holds cloud-app activity, and give a field you'd pivot on. *(CloudAppEvents.)*
10. **Boundary & governance:** Which decisions (app sanctioning, consent revocation, user suspension) require the customer's approval, and who approves them?

B-capstone — Live demonstration

Each attendee, unaided, on a **second** app/alert: opens Discovered apps and ranks by risk (states one disposition), runs one saved hunting query and interprets a result, and names the fitting response action and the approval required.

Part B pass: every attendee meets B-know (≥8/10) and completes B-capstone; the B-env checks pass. Anyone below threshold repeats the relevant lab.

Part C — Proof-of-concept gates (Phase C)

C1 — Audit policy scoped

Procedure: Run `MDCAFULL-PoC.ps1 -Action Enable -WhatIf`; confirm `MDCA PoC Activity Policy – Pilot Users – Audit` is scoped to the pilot group with **no governance action** (audit only). **Pass:** The policy targets the pilot users only and takes no action. **Fail:** see Troubleshooting *Policy scope

or action wrong*.

C2 — Benign signal generated

Procedure: Run `MDCAFULL-PoC.ps1 -Action Signal`; confirm the sample discovery log uploaded and the benign policy-matching activity was performed. **Pass:** Both benign signals generated; no real exfiltration. **Fail:** see Troubleshooting *Signal not generated*.

C3 — Discovery report populated

Procedure: `Cloud apps → Cloud discovery → Dashboard` shows `MDCA PoC Discovery – Pilot – Report`; `MDCAFULL-Validate.ps1 -Check PoC` confirms discovery data. **Pass:** The Shadow IT report populated from the sample log with app risk scores. **Fail:** see Troubleshooting *Discovery report empty*.

C4 — Activity policy matched and alert surfaced

Procedure: Confirm the audit policy matched the benign activity and an alert surfaced (`Cloud apps → Alerts` / **Incidents & alerts**); `MDCAFULL-Validate.ps1 -Check PoC` checks `AlertInfo`. **Pass:** The policy matched in audit mode and an alert was raised (audit still alerts; it just does not govern). **Fail:** see Troubleshooting *Policy did not match* / *No alert surfaced*.

C5 — Activity-log / hunting visibility

Procedure: `Cloud apps → Activity log` shows the benign activity; `MDCAFULL-Validate.ps1 -Check PoC` runs KQL over `CloudAppEvents`. **Pass:** The activity is visible and hunting returns recent rows. **Fail:** allow warm-up; see Troubleshooting *No activity visibility*.

C6 — Scorecard produced

Procedure: Run `MDCAFULL-PoC.ps1 -Action Score` and confirm `MDCA PoC Scorecard – Pilot Users – Validate.json` is written with a pass/fail per criterion. **Pass:** A complete scorecard maps every observation to your criteria. **Fail:** re-run the relevant validate step first.

C7 — Clean teardown

Procedure: Run `MDCAFULL-PoC.ps1 -Action Teardown -WhatIf`, then for real; re-run `MDCAFULL-Validate.ps1 -Check PoC`. **Pass:** The policy, discovery report, and connector are removed and test alerts resolved. **Fail:** see Troubleshooting *Teardown left residue*.

Exit gate

Part A (A1–A8) passes on the pilot **before** you enforce any policy or expand scope. Part B passes to certify the team. Part C (C1–C6) passes to call the PoC conclusive; C7 passes to close it out cleanly. Re-run A1, A2, A6, and A7 after each expansion wave and compare against the saved baseline; keep the scorecard file as the decision record.

6. Troubleshooting Guide — Microsoft Defender for Cloud Apps

Common field issues across all three phases, in **symptom** → **cause** → **fix** form. Run `MDCAFULL-Discovery.ps1` and `MDCAFULL-Validate.ps1` first — most causes surface there. For category-level prerequisites (licensing, roles, network) see `global-requirements-defender.md`.

Cloud apps section missing or blades won't open

Symptom: The **Cloud apps** area is absent in the Defender portal, or its blades error. **Cause:** MDCA is not licensed/active, or the account lacks the role. **Fix:** Confirm the MDCA plan (Admin center → Billing → Licenses; often bundled in E5 Security — *verify current availability*) and that the account holds an MDCA admin / Security Reader role (Requirements 2.1, 2.9). Complete the one-time activation if needed.

Connector not connecting

Symptom: Test A1 fails — a connector in `MDCA App Connectors – Priority SaaS – Prod` shows an error or "Disconnected." **Cause:** The app's admin consent/API credential was not completed or has expired, the connecting account lacks admin rights in that app, or the app's API access is restricted. **Fix:** Re-run the connector's consent flow with an admin of that app; confirm the API credential/token is valid; check the app's own API access settings. Remove and re-add the connector if needed, then re-run `MDCAFULL-Validate.ps1 -Check Connectors`.

No discovery data

Symptom: Test A2 (or Lab 1) fails — **Cloud discovery** → **Dashboard** is empty. **Cause:** No discovery source is configured, the Defender for Endpoint integration is off, pilot devices are not MDE-onboarded, or the log collector is not receiving logs. **Fix:** Confirm a source under **Cloud discovery** → **Settings**; turn on the **Defender for Endpoint** → **Cloud Apps** integration and confirm devices are MDE-onboarded (see the MDE onboarding accelerator); verify the log collector is receiving SWG logs; or upload a snapshot log for the labs/PoC. Allow warm-up, then re-run `MDCAFULL-Validate.ps1 -Check Discovery`.

App governance not inventorying

Symptom: Test A3 (or Labs 2/3) fails — **App governance** shows no OAuth app inventory, or the report policy is absent. **Cause:** App governance is not enabled, the plan does not include it, or it has not finished the initial inventory. **Fix:** Enable app governance under **Cloud apps** → **App governance**; *verify the plan includes app governance in your tenant*; allow the initial inventory to complete, then create `MDCA OAuth App Risk – Tenant – Report`. If it cannot be enabled, run Lab 3 from the **OAuth apps** inventory read-only. Re-run `MDCAFULL-OAuthRisk.ps1`.

App Control not proxying / session policy has no effect

Symptom: Test A5 (or Lab 4) fails — a pilot user's session is not routed through MDCA (no session banner/suffix), or a created session policy does nothing. **Cause:** The Conditional Access policy `MDCA Session Control – Pilot Users – Report` is not enabled/assigned to the pilot users, the app is not supported by App Control, or the session-control setting is not applied. **Fix:** Confirm the CA policy is

On and assigned to `MDCA Pilot Users – Pilot Ring – Pilot`; confirm the app is supported by Conditional Access App Control (*verify current supported apps*); confirm **Session** → **Use Conditional Access App Control** is set. Keep the policy **Monitor only** during enablement. Have the user sign out/in and re-test.

No cloud-app signal

Symptom: Test A6 (or Labs 2/5) fails — `CloudAppEvents` returns no rows. **Cause:** Connectors are newly added and still warming up, discovery has no source, or the account lacks advanced-hunting read. **Fix:** Confirm connectors are connected (A1) and discovery is flowing (A2); allow warm-up (tables populate over hours); confirm the attendee role includes advanced-hunting read (Requirement 2.7); re-run `MDCAFULL-Validate.ps1 -Check Signal`.

`MDCAFULL-OAuthRisk.ps1` lists apps but no permission grants

Symptom: RiskHint is based only on publisher verification; `HighPrivScopes` is blank. **Cause:** `Get-MgServicePrincipalOauth2PermissionGrant` needs `Directory.Read.All` and returns delegated grants only; some app-role (application) permissions are not shown by this call. **Fix:** Confirm `Directory.Read.All` consent. Cross-check high-privilege **application** permissions in **Entra** → **Enterprise applications** → **Permissions**. Treat the RiskHint as a starting sort, not a verdict.

Anomaly-detection alerts absent (Lab 5)

Symptom: No Defender-for-Cloud-Apps anomaly alerts to triage. **Cause:** A quiet tenant, insufficient baseline (anomaly detection learns normal behavior over time), or alerts outside the window. **Fix:** A quiet tenant is fine for teaching — use the runbook descriptions and the `CloudAppEvents` queries to build the investigation skill. Allow the anomaly engine a learning window; widen the time range.

PoC policy scope or action wrong

Symptom: Test C1 fails — the audit policy applies to more than the pilot users, or it has a governance action. **Cause:** The policy filter was not scoped to `MDCA Pilot Users – Pilot Ring – Pilot`, or a governance action was left set. **Fix:** Edit `MDCA PoC Activity Policy – Pilot Users – Audit` — restrict the filter to the pilot group and set **Governance actions = none**. Re-run `MDCAFULL-PoC.ps1 -Action Enable -WhatIf`.

PoC signal not generated / discovery report empty

Symptom: Test C2/C3 fails — the sample log did not upload, the benign activity did not register, or `MDCA PoC Discovery – Pilot – Report` shows no data. **Cause:** The upload used the wrong format/source, the test account is not in the pilot group, or the snapshot report is still processing. **Fix:** Use Microsoft's sample Cloud Discovery log with a matching data source in the snapshot report; confirm the test account is in `MDCA Pilot Users – Pilot Ring – Pilot`; allow processing time; re-run `MDCAFULL-PoC.ps1 -Action Signal`.

PoC policy did not match / no alert surfaced

Symptom: Test C4 fails — the audit policy did not match the benign activity, or matched but no alert appears. **Cause:** The activity did not meet the policy filter/threshold, the connector is not yet returning activity, signal has not propagated, or alerting is off. **Fix:** Confirm the benign action matches the policy's trigger (threshold/activity type); confirm the connector is returning activity; confirm the policy raises an alert on match (alerting is independent of governance actions); allow warm-up and re-run `MDCAFULL-Validate.ps1 -Check PoC`.

PoC teardown left residue

Symptom: After `MDCAFULL-PoC.ps1 -Action Teardown`, the policy, report, or connector remains. **Cause:** Teardown ran in `-WhatIf` only, or an item must be removed in the portal. **Fix:** Run teardown **without** `-WhatIf`; delete the policy, the discovery report `MDCA PoC Discovery – Pilot – Report`, and any PoC connector in `Cloud apps`; resolve the `MDCA-PoC-TESTSIGNAL` alerts; re-confirm with `MDCAFULL-Validate.ps1 -Check PoC`.

Policy too noisy / false positives

Symptom: A monitor policy generates excessive or irrelevant alerts on the pilot. **Cause:** The policy scope or thresholds are too broad, or a legitimate business workflow matches the pattern. **Fix:** Tighten the policy scope to `MDCA Pilot Users – Pilot Ring – Pilot`, adjust thresholds/filters, and add exclusions for known-good workflows. Keep it report-only until the noise is acceptable, then enforce.

Policy enforced or app revoked unexpectedly

Symptom: A pilot user was blocked, or an OAuth app was revoked, during the pilot. **Cause:** A policy was created in enforce mode, or an app-governance policy had auto-revoke enabled, before validation. **Fix:** Reset the policy to **report-only** (Implementation Step A5) and disable auto-revoke; confirm with the security team; re-validate before enforcing. *Enforcement/revocation decisions are the security team's.*

Script cannot authenticate

Symptom: `MDCAFULL-*.ps1` fails on connect or returns access-denied. **Cause:** The Microsoft Graph modules are missing, consent to the required read scopes was not granted, or the account lacks the role. **Fix:** Let the script install the modules (it checks on run); sign in with `Connect-MgGraph` and consent to the read-only scopes; confirm **Security Reader** (or Security Administrator for the PoC enable/teardown). Re-run the script.

Portal menu paths don't match

Symptom: A navigation path in the guide doesn't exist. **Cause:** MDCA settings are consolidating from the classic portal into `security.microsoft.com`; menus move. **Fix:** Use global search for the feature name (e.g. "Cloud Discovery", "OAuth apps", "App governance"). *Verify current navigation in your tenant* and update your copy of the runbook.

7. Best Practices — Microsoft Defender for Cloud Apps

Field-proven guidance for keeping the Defender for Cloud Apps deployment clean, the team sharp, and the proof-of-concept safe and conclusive.

Connect & Govern (Phase A)

Start every policy in report-only, scoped to a pilot

- Create activity, file, session, and access policies in **monitor / report-only** first, scoped to `MDCA Pilot Users – Pilot Ring – Pilot`. Watch behavior, tune out false positives, then enforce. A CASB that blocks legitimate work on day one loses the room.
- Move to enforce for the pilot, confirm, then broaden the Conditional Access scope in waves — never enforce tenant-wide first.

Connect Microsoft 365 first, then priority third-party apps

- The Microsoft 365 connector gives the fastest, richest signal. Add high-value third-party apps (`MDCA App Connectors – Priority SaaS – Prod`) next by data sensitivity, not by ease. Connectors are out-of-band API integrations — they add signal without user impact.

Use the Defender for Endpoint feed for discovery where you can

- If devices are MDE-onboarded, the built-in **Defender for Endpoint** → **Cloud Apps** integration is the lowest-friction Cloud Discovery source — no log collector to maintain. Fall back to a log collector / SWG feed only where MDE is not the source.

Treat OAuth app governance as a standing risk surface

- Risky **OAuth app** consents exfiltrate data with no stolen password. Keep `MDCA OAuth App Risk – Tenant – Report` running, review flagged apps (over-privileged scopes, unverified publishers) on a cadence with `MDCAFULL-OAuthRisk.ps1`, and keep **revocation approval-gated** — it can break legitimate integrations. Revocation decisions belong to the security team.

Enable the team (Phase B)

Teach against the pilot, read-only

- Run the labs against `MDCA Pilot Users – Pilot Ring – Pilot`. The only configuration action is the report-only session policy (facilitator); everything else is discovery, hunting, and reading. Rehearse OAuth revocation and user suspension without executing them against production.

Use consistent object names and save the artifacts

- The `<Purpose> – <Scope> – <Stage>` convention keeps take-home artifacts findable and reusable. The three saved queries, the report-only policy, the two shortlists, and the investigation doc **are** the runbook — keep them and use them to onboard new analysts.

Instill the operating habits

- **Discover first, then sanction deliberately** — rank shadow IT by risk *and* usage; a high-risk app nobody uses is lower priority than a medium-risk app everyone uses.
- **Monitor before you block** — every session/access policy earns enforcement by proving itself report-only first, then stage the block to a ring with a rollback.

• **Triage anomalies with context** — pivot in `CloudAppEvents` for the user's normal pattern before escalating impossible-travel or mass-download alerts.

Prove value (Phase C)

Keep the PoC scoped, monitor-first, and reversible

• Scope the activity policy to **pilot users only** and keep it in **audit mode with no governance action**. Preview every change with `-WhatIf` first (`MDCAFULL-PoC.ps1 -Action Enable/-Action Teardown`).

Benign signal only — never real exfiltration

• Use Microsoft's **sample Cloud Discovery log** and **benign policy-matching activity** exclusively. **Never** exfiltrate real data, connect a malicious app, or grant a risky OAuth consent to "test." A PoC proves the product surfaces the signal; it is not a data-loss exercise.

Set the bar before you run, then tear down completely

• Agree the **evaluation criteria** in writing first and feed them into `MDCAFULL-PoC.ps1 -Action Score` so the verdict is objective. Afterwards, delete the policy, the discovery report, and any PoC connector, resolve the `MDCA-PoC-TESTSIGNAL` alerts, and confirm with `MDCAFULL-Validate.ps1 -Check PoC`. Keep only the scorecard.

Across all phases

Keep enforcing and revoke actions approval-gated until trusted

• Block, suspend-session, revoke-app, and teardown are disruptive or destructive. Leave them `-WhatIf`- or approval-gated during the pilot and early expansion; moving toward automatic enforcement is the security team's decision, made after validation.

Let discovery and hunting tables warm up

• Discovery data and `CloudAppEvents` populate over minutes to hours. Confirm streaming with `MDCAFULL-Validate.ps1 -Check Signal` before judging coverage, and re-baseline with `MDCAFULL-Report.ps1 -SaveBaseline` after each expansion. If session control is unlicensed, record it as "not licensed," not "failed."

Keep licensing and preview facts current

• MDCA, **app governance**, and **Conditional Access App Control** are license-gated and evolve, and App Control supports a changing set of apps. Before relying on a capability, **verify current availability, licensing, and supported apps in your tenant.**

Defer the data-governance decisions to the owners

• What counts as sensitive data, which apps are sanctioned, and acceptable-use policy are **the customer's compliance/security determinations**. This solution delivers the technical CASB onboarding, the operating skills, and the proof — the classification and policy decisions stay with the owners.

Where to go next

Once MDCA is connected, the team is enabled, and value is proven, natural next steps in the catalog:

- The **Cloud-App DLP (Defender for Cloud Apps) — Microsoft Purview** solution to layer data-loss-prevention on the connected apps and move from audit to real governance org-wide.
- The **Microsoft Defender XDR — Unified Onboarding** accelerator (**DEFXDR**) to correlate MDCA with Endpoint, Identity, and Office 365 in one portal.
- The **Microsoft Defender for Endpoint** to establish the MDE feed that powers Cloud Discovery.
- **Entra Conditional Access baseline** and identity-governance solutions to strengthen the access layer that App Control depends on.
- The **Sentinel / SOC-XDR integration** solution to stream MDCA into a SIEM for long-term hunting.

The bottom line: a fully connected, team-operated, value-proven Defender for Cloud Apps — exposing shadow IT, governing apps and OAuth risk, authoring report-only-first policies, and running clean, approval-gated investigations — the whole CASB lifecycle in one package.

8. Deliverables Summary — Microsoft Defender for Cloud Apps

The complete Defender for Cloud Apps lifecycle in one package — **Connect & Govern** → **Enable the team** → **Prove value**.

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box — the three-phase lifecycle
1. Solution Overview	What MDCA is — app connectors, Cloud Discovery, app governance/OAuth policies, session/access control; the three phases; benefits, limitations
2. Requirements Guide	Solution-specific prerequisites with Verify / Implement-if-missing
Global Requirements (Defender)	Category-wide prerequisites, published once
3. Implementation Guide	All three phases — Phase A connect & govern, Phase B the five enablement labs, Phase C the guided PoC — with named objects
5. Testing Guide	Part A deployment validation, Part B enablement knowledge-check, Part C PoC gates
6. Troubleshooting Guide	Field issues and remediation across all phases
7. Best Practices	Field-proven recommendations per phase and where to go next

Deliverables (PowerShell — Microsoft Graph Security / Entra + KQL)

File	What it does					
`MDCAFULL-Discovery.ps1`	Read-only readiness — MDCA activation, plan hints, admin roles, existing app connectors, Cloud Discovery source, app-governance status, Conditional Access availability, MDCA alert count. Makes no changes.					
`MDCAFULL-Validate.ps1`	Targeted validation (`-Check Connectors\	Discovery\	Policy\	Signal\	PoC\	All) — confirms connectors, discovery flow, report-only/pilot-scoped policies, CloudAppEvents` streaming, and the PoC observations (discovery report, policy match, alert). Read-only.

File	What it does					
`MDCAFULL-Report.ps1`	Coverage report by section (`-Section OAuth\	Policies\	All, -SaveBaseline`) — risky OAuth apps, policy inventory/posture, shadow-IT / OAuth shortlist templates, and drift versus a saved baseline. Read-only.			
`MDCAFULL-OAuthRisk.ps1`	Read-only enumeration of OAuth service principals and consent grants with risk hints (high-privilege scopes, unverified publisher, broad consent); prints the **approval-gated** revoke steps as a **preview only** — revokes nothing.					

File	What it does					
`MDCAFULL-PoC.ps1`	`-WhatIf`-gated proof-of-concept helper (`-Action Enable\	Signal\	Score\	Teardown`) — previews readiness and guides the audit-mode policy, generates **benign** test signal (sample discovery log + benign policy-matching activity), scores against your criteria, and cleanly tears the PoC down. Safe preview by default.		
`MDCAFULL-LabQueries.kql`	Advanced-hunting KQL pack — shadow-IT / discovery, connected-app activity, OAuth apps, anomaly investigation, single-user pivot, and the `MDCA Signal Validation` query. Read-only telemetry.					

Take-home runbook artifacts (produced during the labs)

- Saved hunting queries: `Discovery-ShadowIT – CloudApps – Pilot`, `OAuthApps – CloudApps – Pilot`, `Anomaly-Investigation – CloudApps – Pilot`, `MDCA Signal Validation – Tenant – Validate`.
- The report-only session policy `MDCA Session Control – Pilot Users – Report`.
- The `ShadowIT-Shortlist – CloudApps – Pilot` and `OAuthRisk-Shortlist – CloudApps – Pilot`.
- The `Investigation – CloudApps – Pilot` findings document.
- The `MDCA PoC Scorecard – Pilot Users – Validate` decision record.

What the buyer receives

- This solution as a **Word** document and **PDF**.
- All six deliverables (five PowerShell scripts with embedded KQL, plus the standalone KQL pack) in a single ZIP.
- A repeatable process following the model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-scoped, monitor-first onboarding, hands-on team enablement, and a benign, reversible PoC, with enforcing/revoke/teardown actions kept approval-gated or `-WhatIf`-gated until trusted.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A fully connected, team-ready, value-proven Defender for Cloud Apps delivering CASB — connectors streaming from priority SaaS apps, Cloud Discovery surfacing shadow IT, app governance and OAuth policies watching risky consents, session/access policies enforcing real-time control on a pilot scope, a team that can operate all of it with a runbook they keep, and a scorecard that proves the value — the whole lifecycle in one package.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

MDCAFULL-Discovery.ps1

Purpose: Read-only discovery for the complete "Microsoft Defender for Cloud Apps" solution. Reports the signed-in context, licensed MDCA plan hints, admin roles, Conditional Access availability (for App Control), the MDCA alert count, and prints the portal paths for the connector / discovery / app-governance checks that are portal-only. Makes no changes. Use before Phase A onboarding and as the Lab 0 / PoC readiness check. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDCAFULL-Discovery.ps1 Requires the Microsoft.Graph modules (Authentication + Identity.SignIns for CA read). Connects read-only with Connect-MgGraph. EXPECTED RESULTS A readiness picture written to a JSON snapshot: tenant/context, MDCA plan hints, admin roles, CA availability, alert count — the state to confirm before onboarding, enabling, or a PoC. .NOTES White-label. Read-only. Some MDCA settings surface through the portal only — the script prints the exact portal path where an API read is unavailable. Licensing facts are indicative — verify. > [CmdletBinding()] param([string]$OutputPath = ".\MDCAFULL-discovery.json")

How to run: powershell -ExecutionPolicy Bypass -File .\MDCAFULL-Discovery.ps1 Requires the Microsoft.Graph modules (Authentication + Identity.SignIns for CA read). Connects read-only with Connect-MgGraph. EXPECTED RESULTS A readiness picture written to a JSON snapshot: tenant/context, MDCA plan hints, admin roles, CA availability, alert count — the state to confirm before onboarding, enabling, or a PoC. .NOTES White-label. Read-only. Some MDCA settings surface through the portal only — the script prints the exact portal path where an API read is unavailable. Licensing facts are indicative — verify. > [CmdletBinding()] param([string]$OutputPath = ".\MDCAFULL-discovery.json")

MDCAFULL-LabQueries.kql

MDCAFULL-OAuthRisk.ps1

Purpose: Supports Phase A (OAuth governance) and Lab 3 of the complete "Microsoft Defender for Cloud Apps" solution. Enumerates OAuth applications (service principals) and their delegated / app-role permission grants, flags risk hints (broad/high-privilege scopes, unverified publisher, tenant-wide consent), and prints the precise, approval-gated steps to disable / revoke a risky app in the portal. Requests NO write scope and performs NO revocation — keeping the work read-only / preview-first. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDCAFULL-OAuthRisk.ps1 powershell -ExecutionPolicy Bypass -File .\MDCAFULL-OAuthRisk.ps1 -OutputPath .\mdcafull-oauth.json Run as Security Reader / Security Administrator; consent the read-only scopes when prompted. EXPECTED RESULTS A ranked-ish list of OAuth apps with risk hints and a printed step-by-step for the approval-gated revoke. Use it to build 'OAuthRisk-Shortlist - CloudApps - Pilot'. .NOTES White-label. Read-only / preview-first. Requests NO write scopes and revokes nothing. Revoking consent can break a legitimate integration — always approval-gated with the app owner. > [CmdletBinding()] param([string]$OutputPath = ".\mdcafull-oauth.json")

How to run: powershell -ExecutionPolicy Bypass -File .\MDCAFULL-OAuthRisk.ps1 powershell -ExecutionPolicy Bypass -File .\MDCAFULL-OAuthRisk.ps1 -OutputPath .\mdcafull-oauth.json Run as Security Reader / Security Administrator; consent the read-only scopes when prompted. EXPECTED RESULTS A ranked-ish list of OAuth apps with risk hints and a printed step-by-step for the approval-gated revoke. Use it to build 'OAuthRisk-Shortlist - CloudApps - Pilot'. .NOTES White-label. Read-only /

preview-first. Requests NO write scopes and revokes nothing. Revoking consent can break a legitimate integration — always approval-gated with the app owner. > [CmdletBinding()] param([string]$OutputPath = ".\mdcafull-oauth.json")

MDCAFULL-PoC.ps1

Purpose: Phase C of the complete "Microsoft Defender for Cloud Apps" solution. One -Action runs each PoC step, pilot-scoped and audit-mode only: Enable — preview readiness (plan/roles/discovery source/pilot group) and guide the audit-mode activity policy 'MDCA PoC Activity Policy - Pilot Users - Audit' scoped to the pilot group. Governance actions = NONE. -WhatIf previews. Signal — guide BENIGN signal: upload Microsoft's SAMPLE Cloud Discovery log and perform benign policy-matching activity as a test account (tag MDCA-PoC-TESTSIGNAL). No real exfiltration, ever. Score — map observations (discovery populated, policy matched, alert surfaced, activity visible) against your criteria.json and write the scorecard. Teardown — DESTRUCTIVE: delete the audit policy + discovery report + PoC connector and resolve test alerts. -WhatIf-gated; previews by default. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDCAFULL-PoC.ps1 -Action Enable -WhatIf powershell -ExecutionPolicy Bypass -File .\MDCAFULL-PoC.ps1 -Action Signal powershell -ExecutionPolicy Bypass -File .\MDCAFULL-PoC.ps1 -Action Score -CriteriaPath .\criteria.json powershell -ExecutionPolicy Bypass -File .\MDCAFULL-PoC.ps1 -Action Teardown -WhatIf # then without -WhatIf Requires Microsoft.Graph.Authentication; Security Administrator for Enable/Teardown. EXPECTED RESULTS A guided, reversible PoC — readiness + audit policy, benign signal, a scorecard file, and a clean teardown. No enforcement is ever enabled; the teardown is -WhatIf-gated. .NOTES White-label. Preview-first (-WhatIf) for Enable/Teardown. BENIGN signal only (sample log + benign policy-matching activity). Never real exfiltration. Pilot-scoped, audit-mode only. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([ValidateSet('Enable','Signal','Score','Teardown')] [string]$Action = 'Enable', [string]$PilotGroup = 'MDCA Pilot Users - Pilot Ring - Pilot', [string]$PolicyName = 'MDCA PoC Activity Policy - Pilot Users - Audit', [string]$DiscoveryReport = 'MDCA PoC Discovery - Pilot - Report', [string]$Tag = 'MDCA-PoC-TESTSIGNAL', [string]$CriteriaPath = '.\criteria.json', [string]$OutFile = 'MDCA PoC Scorecard - Pilot Users - Validate.json')

How to run: powershell -ExecutionPolicy Bypass -File .\MDCAFULL-PoC.ps1 -Action Enable -WhatIf powershell -ExecutionPolicy Bypass -File .\MDCAFULL-PoC.ps1 -Action Signal powershell -ExecutionPolicy Bypass -File .\MDCAFULL-PoC.ps1 -Action Score -CriteriaPath .\criteria.json powershell -ExecutionPolicy Bypass -File .\MDCAFULL-PoC.ps1 -Action Teardown -WhatIf # then without -WhatIf Requires Microsoft.Graph.Authentication; Security Administrator for Enable/Teardown. EXPECTED RESULTS A guided, reversible PoC — readiness + audit policy, benign signal, a scorecard file, and a clean teardown. No enforcement is ever enabled; the teardown is -WhatIf-gated. .NOTES White-label. Preview-first (-WhatIf) for Enable/Teardown. BENIGN signal only (sample log + benign policy-matching activity). Never real exfiltration. Pilot-scoped, audit-mode only. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([ValidateSet('Enable','Signal','Score','Teardown')] [string]$Action = 'Enable', [string]$PilotGroup = 'MDCA Pilot Users - Pilot Ring - Pilot', [string]$PolicyName = 'MDCA PoC Activity Policy - Pilot Users - Audit', [string]$DiscoveryReport = 'MDCA PoC Discovery - Pilot - Report', [string]$Tag = 'MDCA-PoC-TESTSIGNAL', [string]$CriteriaPath = '.\criteria.json', [string]$OutFile = 'MDCA PoC Scorecard - Pilot Users - Validate.json')

MDCAFULL-Report.ps1

Purpose: Read-only reporting for the complete "Microsoft Defender for Cloud Apps" solution. By section: OAuth — a summary of risky OAuth apps (delegates to the same read as MDCAFULL-OAuthRisk) plus

the 'OAuthRisk-Shortlist - CloudApps - Pilot' template. Policies — MDCA alerts by severity + the policy inventory/posture reminder (portal confirm). All — both, plus the 'ShadowIT-Shortlist - CloudApps - Pilot' template. -SaveBaseline writes the current numbers as a baseline; a later run reports drift against it. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDCAFULL-Report.ps1 -Section All powershell -ExecutionPolicy Bypass -File .\MDCAFULL-Report.ps1 -Section Policies -SaveBaseline -Section accepts: OAuth | Policies | All (default All). Uses Microsoft.Graph read scopes. EXPECTED RESULTS A printed coverage/risk summary and shortlist templates; a JSON snapshot; and, when a baseline exists, the drift (new/resolved alerts, OAuth-risk change) since the last baseline. .NOTES White-label. Read-only. Shortlists are recommendations — app-sanctioning / revocation decisions are the customer's security team's. > [CmdletBinding()] param([ValidateSet('OAuth','Policies','All')] [string]$Section = 'All', [switch]$SaveBaseline, [string]$OutputPath = ".\MDCAFULL-report.json", [string]$BaselinePath = ".\MDCAFULL-baseline.json")

How to run: powershell -ExecutionPolicy Bypass -File .\MDCAFULL-Report.ps1 -Section All powershell -ExecutionPolicy Bypass -File .\MDCAFULL-Report.ps1 -Section Policies -SaveBaseline -Section accepts: OAuth | Policies | All (default All). Uses Microsoft.Graph read scopes. EXPECTED RESULTS A printed coverage/risk summary and shortlist templates; a JSON snapshot; and, when a baseline exists, the drift (new/resolved alerts, OAuth-risk change) since the last baseline. .NOTES White-label. Read-only. Shortlists are recommendations — app-sanctioning / revocation decisions are the customer's security team's. > [CmdletBinding()] param([ValidateSet('OAuth','Policies','All')] [string]$Section = 'All', [switch]$SaveBaseline, [string]$OutputPath = ".\MDCAFULL-report.json", [string]$BaselinePath = ".\MDCAFULL-baseline.json")

MDCAFULL-Validate.ps1

Purpose: Read-only validation for the complete "Microsoft Defender for Cloud Apps" solution. Confirms the piece named by -Check and maps to the Testing Guide gates: Connectors — connectors connected (portal confirm). Discovery — Cloud Discovery flowing (portal confirm). Policy — session/access policies report-only and scoped to the pilot group. Signal — CloudAppEvents streaming (advanced-hunting reminder + alert read). PoC — the PoC observations: discovery report, audit-policy match, alert surfaced. All — every check. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDCAFULL-Validate.ps1 -Check Policy -Check accepts: Connectors | Discovery | Policy | Signal | PoC | All (default All). Uses Microsoft.Graph.Authentication + Graph Security. Reads only; makes no changes. EXPECTED RESULTS PASS/REVIEW lines per check with the object names from the Implementation Guide, so you can confirm each Testing Guide gate before enforcing or expanding beyond the pilot. .NOTES White-label. Read-only. Where a setting is portal-only, the script prints the exact path to confirm manually. Policies must stay report-only until validated. > [CmdletBinding()] param([ValidateSet('Connectors','Discovery','Policy','Signal','PoC','All')] [string]$Check = 'All', [string]$PilotGroup = 'MDCA Pilot Users - Pilot Ring - Pilot', [string]$PoCPolicyName = 'MDCA PoC Activity Policy - Pilot Users - Audit')

How to run: powershell -ExecutionPolicy Bypass -File .\MDCAFULL-Validate.ps1 -Check Policy -Check accepts: Connectors | Discovery | Policy | Signal | PoC | All (default All). Uses Microsoft.Graph.Authentication + Graph Security. Reads only; makes no changes. EXPECTED RESULTS PASS/REVIEW lines per check with the object names from the Implementation Guide, so you can confirm each Testing Guide gate before enforcing or expanding beyond the pilot. .NOTES White-label. Read-only. Where a setting is portal-only, the script prints the exact path to confirm manually. Policies must stay report-only until validated. > [CmdletBinding()] param([ValidateSet('Connectors','Discovery','Policy','Signal','PoC','All')] [string]$Check = 'All', [string]$PilotGroup =

'MDCA Pilot Users - Pilot Ring - Pilot', [string]$PoCPolicyName = 'MDCA PoC Activity Policy - Pilot Users - Audit')